Sorting Algorithms

1. Introduction to Sorting

- Sorting is the process of arranging data in a specific order.
- Common sorting orders:
  - Ascending Order: Smallest to Largest.
  - Descending Order: Largest to Smallest.
- Sorting is useful for efficient searching, data analysis and data organization.
- Common sorting algorithms:
  1. Selection Sort
  2. Bubble Sort
  3. Insertion Sort
  4. Merge Sort
  5. Quick Sort
  6. Heap Sort

SUNBEAM

---

2. Selection Sort

- Selection Sort repeatedly selects the smallest element from the unsorted portion and places it at the correct position.
- The array is divided into:
  - Sorted portion.
  - Unsorted portion.
- In every pass, the minimum element from the unsorted portion is selected.
- It is swapped with the first element of the unsorted portion.
- Number of passes for `n` elements: `n - 1`.

Important Points

- Performs minimum number of swaps compared to common elementary sorting algorithms.
- Does not adapt to already sorted input.
- Standard Selection Sort is not stable.
- It is an in-place sorting algorithm.

Complexity

- Best Case: $O(n^2)$
- Average Case: $O(n^2)$
- Worst Case: $O(n^2)$
- Auxiliary Space: O(1)

---

3. Bubble Sort

- Bubble Sort repeatedly compares adjacent elements and swaps them if they are in the wrong order.
- After every pass, the largest unsorted element reaches its correct position at the end.
- The process continues until the array is sorted.
- An optimized version uses a `swapped` flag.
- If no swapping occurs during a pass, the array is already sorted and the algorithm terminates.

Important Points

- Also called Exchange Sort.
- Largest element moves towards the end in ascending order.
- Stable sorting algorithm.
- In-place sorting algorithm.
- Optimized Bubble Sort performs efficiently on already sorted arrays.

Complexity

- Best Case: O(n) (optimized version with swapped flag).

SUNBEAM

- Average Case: $O(n^2)$
- Worst Case: $O(n^2)$
- Auxiliary Space: O(1)

---

4. Insertion Sort

- Insertion Sort builds the sorted array one element at a time.
- It divides the array into:
  - Sorted portion.
  - Unsorted portion.
- Each element from the unsorted portion is picked and inserted at its correct position in the sorted portion.
- Similar to arranging playing cards in hand.

Important Points

- Efficient for small datasets.
- Efficient for nearly sorted arrays.
- Stable sorting algorithm.
- In-place sorting algorithm.
- Adaptive algorithm: performs fewer operations when data is nearly sorted.

Complexity

- Best Case: O(n) (already sorted array).
- Average Case: $O(n^2)$
- Worst Case: $O(n^2)$
- Auxiliary Space: O(1)

---

5. Merge Sort

SUNBEAM

- Merge Sort follows the **Divide and Conquer** technique.
- It divides the array into two halves recursively.
- Each half is sorted independently.
- Finally, both sorted halves are merged into a single sorted array.

Working Steps

1. Divide the array into two halves.
2. Continue dividing until each subarray contains one element.
3. Merge smaller sorted subarrays.
4. Continue merging until the complete array is sorted.

Important Points

- Follows Divide and Conquer.
- Recursively divides the array into smaller parts.
- Stable sorting algorithm.
- Standard array implementation is not in-place.
- Suitable for large datasets.
- Performs consistently irrespective of input order.

Complexity

- Best Case: O(n log n)
- Average Case: O(n log n)
- Worst Case: O(n log n)
- Auxiliary Space: O(n) for array implementation.
- Recursion Stack: O(log n)

---

6. Quick Sort

SUNBEAM

- Quick Sort follows the **Divide and Conquer** technique.
- It selects an element called the **Pivot**.
- Elements smaller than the pivot are placed on one side.
- Elements greater than the pivot are placed on the other side.
- This process is called Partitioning.
- The same process is recursively applied to the left and right partitions.

Working Steps

1. Select a pivot element.
2. Partition the array around the pivot.
3. Place the pivot in its correct sorted position.
4. Recursively sort the left partition.
5. Recursively sort the right partition.

Important Points

- Follows Divide and Conquer.
- Performance depends on pivot selection and partitioning.
- Generally faster in practice due to good cache locality and low auxiliary memory usage.
- Standard in-place partitioning implementations are not stable.
- In-place sorting algorithm in its commonly used implementations.
- Worst case occurs when partitioning repeatedly produces highly unbalanced partitions.

Complexity

- Best Case: O(n log n)
- Average Case: O(n log n)
- Worst Case: $O(n^2)$
- Average Recursion Stack: O(log n)
- Worst-case Recursion Stack: O(n)

SUNBEAM

7. Heap Sort

- Heap Sort uses a Binary Heap to sort elements.
- It generally uses a Max Heap for ascending order.
- First, a Max Heap is constructed from the given array.
- The root contains the maximum element.
- The root is swapped with the last element.
- The heap size is reduced and Heapify Down is performed.
- The process continues until all elements are sorted.

Working Steps

1. Build a Max Heap.
2. Swap the root with the last element.
3. Reduce the heap size by one.
4. Apply Heapify Down to restore the Max Heap property.
5. Repeat until the heap size becomes one.

Important Points

- Uses Binary Heap.
- Follows the heap-based sorting approach.
- In-place sorting algorithm.
- Standard Heap Sort is not stable.
- Provides guaranteed O(n log n) time complexity.
- Does not require an additional array of size n.

Complexity

- Best Case: O(n log n)
- Average Case: O(n log n)
- Worst Case: O(n log n)

SUNBEAM

- Auxiliary Space: O(1) for iterative Heapify.

8. Comparison of Sorting Algorithms

Time Complexity Comparison

Algorithm	Best Case	Average Case	Worst Case
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Bubble Sort (Optimized)	O(n)	$O(n^2)$	$O(n^2)$
Insertion Sort	O(n)	$O(n^2)$	$O(n^2)$
Merge Sort	O(n log n)	O(n log n)	O(n log n)
Quick Sort	O(n log n)	O(n log n)	$O(n^2)$
Heap Sort	O(n log n)	O(n log n)	O(n log n)

Space Complexity Comparison

Algorithm	Auxiliary Space	In-place?
Selection Sort	O(1)	Yes
Bubble Sort	O(1)	Yes
Insertion Sort	O(1)	Yes
Merge Sort	O(n)	No (standard array implementation)
Quick Sort	O(log n) average, O(n) worst	Yes (standard in-place partitioning)
Heap Sort	O(1)	Yes

SUNBEAM

9. In-place vs Out-of-place Sorting

In-place Sorting

- Sorting is performed within the original data structure.
- Requires very little additional memory.
- Usually uses O(1) auxiliary space.
- Some algorithms may use a recursion stack.
- Examples:
  - Selection Sort
  - Bubble Sort
  - Insertion Sort
  - Heap Sort
  - Quick Sort (standard in-place partitioning)

Out-of-place Sorting

- Requires additional memory to perform sorting.
- May create temporary arrays or data structures.
- Generally requires more auxiliary space.
- Example:
  - Merge Sort (standard array implementation).

Difference

Feature	In-place	Out-of-place
Memory	Very little extra memory	Additional memory required
Auxiliary Space	Usually O(1)	Usually more than O(1)
Original Structure	Modified directly	May use temporary structures

SUNBEAM

Feature	In-place	Out-of-place
Example	Heap Sort	Merge Sort

Note: In-place does not necessarily mean zero extra memory. Recursion stack and other auxiliary variables may still be required.

---

10. Stable vs Unstable Sorting

Stable Sorting

- A sorting algorithm is stable if it preserves the relative order of elements having equal keys.
- If two elements have the same sorting key, their original order remains unchanged.

Example:

Original order:

```
(A, 10), (B, 20), (C, 10)
```

After stable sorting by value:

```
(A, 10), (C, 10), (B, 20)
```

- A appears before C because both have the same key and A appeared first originally.

Stable Algorithms

- Bubble Sort
- Insertion Sort
- Merge Sort (standard stable merging implementation)

Unstable Sorting

- A sorting algorithm is unstable if it may change the relative order of elements having equal keys.

SUNBEAM

- Equal elements may appear in a different order after sorting.

Unstable Algorithms

- Selection Sort (standard implementation)
- Quick Sort (standard in-place implementation)
- Heap Sort

Difference

Feature	Stable Sorting	Unstable Sorting
Equal elements	Preserves relative order	May change relative order
Duplicate keys	Original order maintained	Original order not guaranteed
Example	Bubble, Insertion, Merge	Selection, Quick, Heap

11. Important MCQ Points

- Selection Sort always takes $O(n^2)$ time.
- Selection Sort performs at most O(n) swaps.
- Bubble Sort moves the largest element to the end in every pass.
- Optimized Bubble Sort has O(n) best-case time complexity.
- Insertion Sort is efficient for nearly sorted arrays.
- Insertion Sort is stable and in-place.
- Merge Sort follows Divide and Conquer.
- Merge Sort always takes O(n log n) time in standard implementations.
- Merge Sort is stable but requires O(n) auxiliary space for arrays.
- Quick Sort uses a Pivot and Partitioning.
- Quick Sort has $O(n^2)$ worst-case time complexity.
- Quick Sort is generally unstable.

SUNBEAM

- Heap Sort uses a Binary Heap.
- Heap Sort provides O(n log n) worst-case time complexity.
- Heap Sort is in-place but unstable.
- Stable sorting preserves the relative order of equal elements.
- In-place sorting uses very little additional memory.
- Out-of-place sorting requires additional memory.
- Among these algorithms, Merge Sort provides guaranteed O(n log n) time and stability.
- Heap Sort provides guaranteed O(n log n) time with O(1) auxiliary space.
- Insertion Sort is a good choice for small or nearly sorted datasets.

SUNBEAM